

Pawspa DevOps Project Plan & To-Do List

Project Name

Pawspa Web Application DevOps CI/CD Project

Selected Repository

Original source repository:

`https://github.com/KimToan2501/Du-An-Web/tree/Pawspa`

Recommended new repository name:

`pawspa-devops`

Recommended repository description:

A PHP-based Pawspa web application integrated with Docker, MySQL, Jenkins CI/CD, and automated deployment.

1. Project Overview

1.1 Project Context

This project uses the **Pawspa** web application from the `Pawspa` branch of the repository `KimToan2501/Du-An-Web`.

The original application is a PHP-based web application using:

- PHP
- Composer
- MySQL database
- SQL seed files
- PHP built-in development server
- Environment configuration

The goal of this DevOps project is not to modify the main business logic of the application, but to apply DevOps practices to make the project easier to build, test, package, and deploy automatically.

2. Main DevOps Objectives

2.1 Core Objectives

The project will implement the following DevOps components:

- Source code management with GitHub
- Local application setup
- Environment variable management
- Dockerization of the PHP application
- MySQL database containerization
- Docker Compose orchestration
- Jenkins CI/CD pipeline
- Docker image build and push
- Automated deployment using Docker Compose
- Basic validation and health checking
- Project documentation

2.2 Final Expected Result

After completion, the workflow should be:

```
Developer pushes code to GitHub
↓
Jenkins is triggered automatically
↓
Jenkins checks out source code
↓
Jenkins validates Composer configuration
↓
Jenkins checks PHP syntax
↓
Jenkins builds Docker image
↓
Jenkins pushes image to Docker Hub
↓
Jenkins deploys the application using Docker Compose
↓
Pawspa app runs with MySQL database
```

3. Target Architecture

3.1 Basic Architecture

```
GitHub Repository
  ↓
Jenkins CI/CD Pipeline
  ↓
Docker Image Build
  ↓
Docker Hub Registry
  ↓
Deployment Server
  ↓
Docker Compose Stack
  ↓
PHP App + MySQL Database
```

3.2 Docker Compose Services

The final deployment should include:

pawspa-app	PHP web application container
pawspa-mysql	MySQL database container
phpmyadmin	Optional database management UI

Optional advanced setup:

nginx	Reverse proxy / production web server
-------	---------------------------------------

4. Tools and Technologies

4.1 Development Tools

- Git
- GitHub
- VS Code
- PHP
- Composer

- MySQL

4.2 DevOps Tools

- Docker Desktop
- Docker Compose
- Jenkins
- Docker Hub
- GitHub Webhook

4.3 Optional Tools

- phpMyAdmin
 - Nginx
 - SonarQube
 - Trivy image scanner
 - Prometheus / Grafana
-

5. Project Deliverables

5.1 Required Deliverables

The final project should include:

```
pawspa-devops/  
├ docs/  
├ public/  
├ src/  
├ Dockerfile  
├ .dockerignore  
├ docker-compose.yml  
├ Jenkinsfile  
├ .env.example  
├ README.md  
├ composer.json  
├ composer.lock  
├ lab.sql  
├ seed-data.sql  
└ server.php
```

5.2 Documentation Deliverables

- Project README

- DevOps architecture diagram
 - CI/CD pipeline explanation
 - Setup guide
 - Deployment guide
 - Troubleshooting section
 - Presentation slides
-

6. Phase 1 — Repository Preparation

Goal

Copy the selected Pawspa app into your own GitHub repository and prepare it for DevOps implementation.

To-Do List

- ☐ Clone the original repository with the `Pawspa` branch:

```
git clone -b Pawspa https://github.com/KimToan2501/Du-An-Web.git pawspa-devops
cd pawspa-devops
```

- ☐ Remove old Git remote:

```
git remote remove origin
```

- ☐ Create a new GitHub repository named:

```
pawspa-devops
```

- ☐ Add your new repository as remote:

```
git remote add origin https://github.com/YOUR_USERNAME/pawspa-devops.git
```

- ☐ Push source code to your repository:

```
git branch -M main
git push -u origin main
```

- ☐ Create a new branch for DevOps work:

```
git checkout -b devops-setup
```

Expected Output

- Source code is available in your own GitHub repository.
 - `main` branch contains original stable source code.
 - `devops-setup` branch is used for Docker/Jenkins implementation.
-

7. Phase 2 — Local Application Setup

Goal

Run the application locally before applying DevOps tools.

To-Do List

- ☐ Install PHP.
- ☐ Install Composer.
- ☐ Install MySQL or use XAMPP.
- ☐ Install project dependencies:

```
composer install
```

- ☐ Create local database in MySQL, for example:

```
pawspa
```

- ☐ Import database schema:

```
lab.sql
```

- ☐ Import seed data:

```
seed-data.sql
```

- ☐ Configure `.env` file if the application requires it.
- ☐ Run the application locally:

```
composer run dev
```

- ☐ Open the application in browser:

```
http://localhost:8000
```

Validation Checklist

- ☐ Homepage loads successfully.
- ☐ Database connection works.
- ☐ Login page works.
- ☐ Admin account can log in.
- ☐ Main features are accessible.

Expected Output

The Pawspa app runs successfully on the local machine.

8. Phase 3 — Environment Configuration

Goal

Separate configuration from source code and prepare the app for containerized deployment.

To-Do List

- ☐ Create `.env.example` file.
- ☐ Do not commit real secrets.
- ☐ Add `.env` to `.gitignore`.
- ☐ Define database variables:

```
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=pawspa
DB_USERNAME=root
DB_PASSWORD=root
```

- ☐ Add application variables if needed:

```
APP_ENV=development
APP_PORT=8000
```

- ☐ Add mail variables if needed:

```
MAIL_HOST=smtp.example.com
MAIL_PORT=587
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_FROM=
```

Expected Output

The project has a clean environment configuration structure.

9. Phase 4 — Dockerization

Goal

Create a Docker image for the PHP application.

To-Do List

- [] Create `Dockerfile`.
- [] Install PHP extensions needed for MySQL.
- [] Install Composer dependencies.
- [] Copy application source code into image.
- [] Expose application port.
- [] Run the PHP application inside the container.

Suggested Simple Dockerfile

```
FROM php:8.2-cli

WORKDIR /var/www/html

RUN apt-get update && apt-get install -y
    unzip
    git
    libzip-dev
    && docker-php-ext-install pdo pdo_mysql zip

COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

COPY composer.json composer.lock ./
RUN composer install --no-interaction --prefer-dist
```



```
COPY . .
```

```
EXPOSE 8000
```

```
CMD ["php", "-S", "0.0.0.0:8000", "-t", "public", "server.php"]
```

To-Do List for `.dockerignore`

- ☐ Create `.dockerignore` file:

```
.git  
.env  
vendor  
node_modules  
*.log  
Dockerfile  
docker-compose.yml  
README.md
```

Docker Build Test

```
docker build -t pawspa-app .
```

Docker Run Test

```
docker run -p 8000:8000 pawspa-app
```

Validation Checklist

- ☐ Docker image builds successfully.
- ☐ Container starts successfully.
- ☐ App is accessible on `http://localhost:8000`.

Expected Output

The Pawspa application can run inside a Docker container.

10. Phase 5 — Docker Compose Setup

Goal

Use Docker Compose to run the PHP application and MySQL database together.

To-Do List

- [] Create `docker-compose.yml`.
- [] Add `app` service.
- [] Add `mysql` service.
- [] Add MySQL volume for persistent storage.
- [] Add SQL initialization scripts.
- [] Add optional `phpmyadmin` service.

Suggested `docker-compose.yml`

```
version: "3.8"

services:
  app:
    build: .
    container_name: pawspa-app
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - mysql
    networks:
      - pawspa-network

  mysql:
    image: mysql:8.0
    container_name: pawspa-mysql
    restart: always
    environment:
      MYSQL_DATABASE: pawspa
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql
      - ./lab.sql:/docker-entrypoint-initdb.d/1-lab.sql
      - ./seed-data.sql:/docker-entrypoint-initdb.d/2-seed-data.sql
```

```
networks:
  - pawspa-network

phpmyadmin:
  image: phpmyadmin/phpmyadmin
  container_name: pawspa-phpmyadmin
  restart: always
  ports:
    - "8080:80"
  environment:
    PMA_HOST: mysql
    PMA_PORT: 3306
  depends_on:
    - mysql
  networks:
    - pawspa-network

volumes:
  mysql_data:

networks:
  pawspa-network:
    driver: bridge
```

Docker Compose Commands

Start services:

```
docker compose up --build
```

Run in background:

```
docker compose up -d --build
```

Stop services:

```
docker compose down
```

Reset database volume:

```
docker compose down -v
```

Validation Checklist

- ☐ `app` container is running.
 - ☐ `mysql` container is running.
 - ☐ `phpmyadmin` container is running.
 - ☐ App connects to MySQL successfully.
 - ☐ Database tables are imported successfully.
 - ☐ App works at `http://localhost:8000`.
 - ☐ phpMyAdmin works at `http://localhost:8080`.
-

11. Phase 6 — Basic CI Checks

Goal

Add basic checks to ensure the PHP project is valid before deployment.

To-Do List

- ☐ Validate Composer file:

```
composer validate
```

- ☐ Install dependencies:

```
composer install --no-interaction --prefer-dist
```

- ☐ Check PHP syntax:

```
find src public -name "*.php" -print0 | xargs -0 -n1 php -l
```

For Windows PowerShell:

```
Get-ChildItem -Recurse -Include *.php src, public | ForEach-Object { php -l  
$_.FullName }
```

- ☐ Build Docker image:

```
docker build -t pawspa-app .
```

Expected Output

The application passes basic CI validation.

12. Phase 7 — Jenkins Setup

Goal

Install and configure Jenkins for CI/CD automation.

To-Do List

- ☐ Install Jenkins.
- ☐ Install Docker.
- ☐ Make sure Jenkins can run Docker commands.
- ☐ Install Jenkins plugins:
 - Git Plugin
 - Pipeline Plugin
 - Docker Pipeline Plugin
 - Credentials Binding Plugin
 - GitHub Integration Plugin

Jenkins Credentials

Create the following credentials:

```
Credential ID: dockerhub-credentials
Type: Username with password
Purpose: Login to Docker Hub
```

Optional:

```
Credential ID: pawspa-env-file
Type: Secret file
Purpose: Store production .env file
```

Validation Checklist

- ☐ Jenkins can access GitHub repository.
- ☐ Jenkins can run Docker commands.
- ☐ Jenkins can login to Docker Hub.

13. Phase 8 — Jenkinsfile CI/CD Pipeline

Goal

Create a Jenkins pipeline that automates build, validation, Docker image creation, image push, and deployment.

To-Do List

- [] Create `Jenkinsfile` in project root.
- [] Add checkout stage.
- [] Add Composer validation stage.
- [] Add PHP syntax check stage.
- [] Add Docker build stage.
- [] Add Docker push stage.
- [] Add deployment stage.

Suggested Jenkinsfile

```
pipeline {
    agent any

    environment {
        IMAGE_NAME = "YOUR_DOCKERHUB_USERNAME/pawspa-devops"
        IMAGE_TAG = "${BUILD_NUMBER}"
        CONTAINER_NAME = "pawspa-app"
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Composer Validate') {
            steps {
                sh 'composer validate || true'
            }
        }

        stage('PHP Syntax Check') {
            steps {
                sh 'find src public -name "*.php" -print0 | xargs -0 -n1 php -l'
```

```

    }
  }

  stage('Docker Build') {
    steps {
      sh 'docker build -t $IMAGE_NAME:$IMAGE_TAG -t
$IMAGE_NAME:latest .'
    }
  }

  stage('Docker Login') {
    steps {
      withCredentials([usernamePassword(credentialsId: 'dockerhub-
credentials', usernameVariable: 'DOCKER_USER', passwordVariable:
'DOCKER_PASS')]) {
        sh 'echo $DOCKER_PASS | docker login -u $DOCKER_USER --
password-stdin'
      }
    }
  }

  stage('Docker Push') {
    steps {
      sh 'docker push $IMAGE_NAME:$IMAGE_TAG'
      sh 'docker push $IMAGE_NAME:latest'
    }
  }

  stage('Deploy') {
    steps {
      sh 'docker compose down || true'
      sh 'docker compose up -d --build'
    }
  }
}

post {
  success {
    echo 'Pipeline completed successfully. Pawspa has been deployed.'
  }
  failure {
    echo 'Pipeline failed. Please check logs.'
  }
}
}

```

Note for Windows Jenkins

If Jenkins runs on Windows, replace `sh` with `bat` commands.

Example:

```
bat 'docker build -t %IMAGE_NAME%:%IMAGE_TAG% .'
```

Expected Output

Jenkins automatically builds and deploys the Pawspa application.

14. Phase 9 — GitHub Webhook

Goal

Trigger Jenkins automatically whenever new code is pushed to GitHub.

To-Do List

- ☐ Open GitHub repository.
- ☐ Go to Settings.
- ☐ Go to Webhooks.
- ☐ Add webhook.
- ☐ Payload URL:

```
http://YOUR_JENKINS_URL/github-webhook/
```

- ☐ Content type:

```
application/json
```

- ☐ Event trigger:

```
Just the push event
```

- ☐ Save webhook.
- ☐ Test by pushing a commit.

Validation Checklist

- ☐ GitHub webhook returns success response.
 - ☐ Jenkins job starts automatically after push.
 - ☐ Pipeline completes successfully.
-

15. Phase 10 — Deployment Validation

Goal

Confirm the deployed application works correctly after CI/CD pipeline runs.

To-Do List

- ☐ Check running containers:

```
docker ps
```

- ☐ Check app logs:

```
docker logs pawspa-app
```

- ☐ Check MySQL logs:

```
docker logs pawspa-mysql
```

- ☐ Open app:

```
http://localhost:8000
```

- ☐ Open phpMyAdmin:

```
http://localhost:8080
```

- ☐ Test login.
- ☐ Test database-related features.
- ☐ Test admin account.

Expected Output

The deployed system runs successfully after the Jenkins pipeline.

16. Optional Enhancement 1 — Nginx Reverse Proxy

Goal

Improve production deployment architecture.

To-Do List

- ☐ Add Nginx service.
- ☐ Route requests from port 80 to app service.
- ☐ Add `nginx.conf`.
- ☐ Update `docker-compose.yml`.

Target Architecture

```
Browser
↓
Nginx :80
↓
PHP App :8000
↓
MySQL :3306
```

Why This Adds Value

- More production-like deployment
 - Cleaner public port
 - Easier to explain reverse proxy
 - Better DevOps presentation
-

17. Optional Enhancement 2 — Security Scanning

Goal

Add security checks to CI/CD pipeline.

To-Do List

- ☐ Install Trivy.
- ☐ Scan Docker image:

```
trivy image YOUR_DOCKERHUB_USERNAME/pawspa-devops:latest
```

- ☐ Add Trivy scan stage in Jenkins.
- ☐ Report vulnerabilities in presentation.

Why This Adds Value

- Demonstrates DevSecOps awareness
 - Improves project quality
 - Adds professional CI/CD stage
-

18. Optional Enhancement 3 — SonarQube

Goal

Add static code quality analysis.

To-Do List

- ☐ Run SonarQube with Docker.
- ☐ Configure Jenkins SonarQube plugin.
- ☐ Add SonarQube analysis stage.
- ☐ View code quality dashboard.

Why This Adds Value

- Demonstrates code quality control
 - Makes pipeline look more professional
 - Useful for presentation screenshots
-

19. Git Branch Strategy

Recommended Branches

main	Stable version
devops-setup	Docker + Jenkins + deployment setup
feature/*	Optional feature changes
fix/*	Bug fixes

Workflow

```
Create changes in devops-setup
↓
Test locally
↓
Push to GitHub
↓
Jenkins pipeline runs
↓
Merge to main after successful build
```

20. Suggested Timeline

Day 1 — Repository Setup

- Clone original repo
- Push to your own GitHub repository
- Create `devops-setup` branch
- Review project structure

Day 2 — Local Setup

- Install PHP, Composer, MySQL
- Import database
- Run app locally
- Document setup issues

Day 3 — Dockerization

- Write `Dockerfile`

- Write `.dockerignore`
- Build Docker image
- Run app container

Day 4 — Docker Compose

- Add MySQL service
- Add phpMyAdmin service
- Add database volume
- Auto-import SQL files
- Validate full stack

Day 5 — Jenkins Setup

- Install Jenkins
- Install plugins
- Add Docker Hub credentials
- Create Jenkins pipeline job

Day 6 — Jenkinsfile & CI/CD

- Write Jenkinsfile
- Add Composer validation
- Add PHP syntax check
- Add Docker build/push
- Add deploy stage

Day 7 — Webhook & Final Demo

- Configure GitHub webhook
- Test auto build on push
- Take screenshots
- Prepare slides
- Final demo rehearsal

21. Common Problems and Solutions

Problem 1: Composer Not Found

Cause

Composer is not installed or not added to PATH.

Fix

Install Composer and restart terminal.

Check:

```
composer -V
```

Problem 2: PHP Not Found

Cause

PHP is not installed or not added to PATH.

Fix

Install PHP or XAMPP and add PHP path to environment variables.

Check:

```
php -v
```

Problem 3: MySQL Connection Error

Cause

Wrong database host or credentials.

Fix

For Docker Compose, use:

```
DB_HOST=mysql  
DB_PORT=3306
```

Do not use `localhost` inside Docker container to connect to MySQL service.

Problem 4: SQL Files Not Imported

Cause

MySQL volume already exists, so init scripts do not run again.

Fix

Reset volume:

```
docker compose down -v
docker compose up -d --build
```

Problem 5: Jenkins Cannot Run Docker

Cause

Jenkins user does not have Docker permission.

Linux Fix

```
sudo usermod -aG docker jenkins
sudo systemctl restart jenkins
```

Windows Fix

Run Jenkins service with a user that has Docker Desktop access.

Problem 6: Port Already in Use

Cause

Another service is using port 8000, 8080, or 3307.

Fix

Change ports in `docker-compose.yml`.

Example:

```
ports:  
- "8001:8000"
```

22. Presentation Outline

Slide 1 — Title

Pawspa Web Application DevOps CI/CD Project

Slide 2 — Project Background

- Original app: Pawspa PHP web app
- Source: GitHub repository
- Goal: Apply DevOps practices

Slide 3 — Technology Stack

- PHP
- Composer
- MySQL
- Docker
- Jenkins
- Docker Hub

Slide 4 — DevOps Architecture

Show pipeline:

```
GitHub → Jenkins → Docker Build → Docker Hub → Docker Compose Deploy
```

Slide 5 — Dockerization

Explain:

- Dockerfile
- .dockerignore
- PHP container
- MySQL container

Slide 6 — Docker Compose

Explain services:

- app
- mysql
- phpmyadmin

Slide 7 — Jenkins Pipeline

Explain stages:

- Checkout
- Composer validate
- PHP syntax check
- Docker build
- Docker push
- Deploy

Slide 8 — CI/CD Demo

Show:

- Push code
- Jenkins auto trigger
- Pipeline success
- App running

Slide 9 — Challenges

Possible challenges:

- Database connection in Docker
- Environment variables
- Jenkins Docker permission
- SQL import with volumes

Slide 10 — Future Improvements

- Nginx reverse proxy
 - SonarQube
 - Trivy security scan
 - Monitoring
 - Cloud deployment
-

23. Final Success Criteria

Minimum Requirements

- ☐ App source is in your own GitHub repo.
- ☐ App runs locally.
- ☐ App runs with Docker.
- ☐ MySQL runs with Docker Compose.
- ☐ Jenkins pipeline builds the project.
- ☐ Jenkins builds Docker image.
- ☐ Jenkins deploys the application.
- ☐ README documents the process.

Good Score Requirements

- ☐ Docker image pushed to Docker Hub.
- ☐ GitHub webhook triggers Jenkins automatically.
- ☐ Database auto-initializes from SQL files.
- ☐ phpMyAdmin included for database verification.
- ☐ Clear architecture diagram in slides.

High Score Requirements

- ☐ Nginx reverse proxy.
- ☐ Trivy security scanning.
- ☐ SonarQube code quality check.
- ☐ Health check in Docker Compose.
- ☐ Deployment to VM/cloud server.

24. Final Recommended Implementation Scope

Recommended Scope for This Course

The best scope is:

```
GitHub
+ Dockerfile
+ docker-compose.yml
+ MySQL container
+ phpMyAdmin
+ Jenkinsfile
+ Docker Hub push
```

- + Auto deploy by Jenkins
- + GitHub webhook

This scope is realistic, complete, and suitable for a DevOps course project.

Avoid Overcomplicating

Do not add Kubernetes unless required. Do not rewrite the whole application. Do not spend too much time changing UI/features.

Focus on:

- Automation
- Containerization
- CI/CD
- Deployment
- Documentation

25. Final Workflow Summary

1. Clone Pawspa repo
2. Push to personal GitHub repo
3. Run app locally
4. Create .env.example
5. Write Dockerfile
6. Write docker-compose.yml
7. Run app + MySQL with Docker Compose
8. Create Jenkinsfile
9. Configure Jenkins credentials
10. Build Docker image in Jenkins
11. Push image to Docker Hub
12. Deploy with Docker Compose
13. Configure GitHub webhook
14. Prepare README and slides
15. Demo full CI/CD pipeline

26. Key Message for Presentation

This project demonstrates how DevOps practices can transform a traditional PHP web application into an automated, containerized, and deployable system using Docker, Jenkins, Docker Compose, GitHub, and Docker Hub.